

Iteration Guide — Site Inspection App

How to update, extend and redeploy the app

App v1.3 · Form 4.2 · Project 2.1 — Site Assessment Live app: <https://ds-js1.github.io/dryspace-inspect/>

What changed in v1.3. The app is now in a **git repository** with an **automated test suite**, so "save as a new version folder" and the manual test checklist are both retired. Governance moved out of this document into three that are loaded automatically — see §0.

0. Read these first

This guide covers **how to change the code**. It deliberately no longer covers what *e*/se must be updated when you do — that drifted, and now lives elsewhere:

Document	Covers
CLAUDE.md	The non-negotiables. Loaded automatically at the start of every session
05_Release Protocol.md	What to update and when, tiered by change size
docs/DECISION-LOG.md	What was decided and why — read before proposing anything architectural

The decision log matters most. Many questions that look open are already settled, and §3 of it lists decisions that were *reversed* — reverting to any of those reintroduces a known bug.

1. Where everything lives

Source of truth is the repository working folder:

```
01_Contact to Contract\Dryspace Inspection App v1.4\  
  index.html          the app – form content, record layer, UI, handover  
  ds-media-sync.js    renditions, naming, upload queue, state machine,  
                      interrupted-upload recovery, readability pre-flight  
  ds-sharepoint.js    Microsoft Graph transport – upload/verify, and from  
                      v1.4 list/download/move/putSmall, all with deadlines  
  ds-auth.js          OAuth 2.0 PKCE sign-in, with its own request deadline  
  sw.js               offline caching + update detection  
  tests.html          366 assertions – run before and after any change  
  diagnostics.html    run on a failing device; names the failing hypothesis  
  manifest.webmanifest, icons
```

Git internals live **outside** this folder at C:\Users\jamie\dev\.git-dryspace-inspect, because the folder sits in a OneDrive-synced SharePoint library and syncing .git corrupts it. The .git entry here is a pointer file — do not delete it.

Deployment: GitHub account **DS-JS1**, repository **dryspace-inspect**, live via GitHub Pages from **main**.

Folder naming changed in v1.3. The folder matches the **minor** version (v1.3), not every patch. Git holds each version and tags each release, so copying the whole folder for a typo fix creates snapshots nobody reads.

2. Starting an update session

Point the session at the v1.3 folder so `CLAUDE.md` loads automatically. If that is awkward, start with:

```
Read CLAUDE.md and docs/DECISION-LOG.md in <path>, then ...
```

State the change in your usual list format. Reference fields by their on-form reference number (e.g. 03.4) and their label.

Example prompt

```
In index.html:
(1) Section 05 – add checkbox "Perched water table confirmed by scan" to
    Contributing structural / design factors;
(2) Section 01 – add a B2C/B2B client type dropdown after Client name(s).

Assign data-fid values to the new controls, run tests.html before and after,
update the changelog, bump the versions, and commit.
```

3. The structural rules

Every control needs a permanent `data-fid`

```
<input type="text" data-fid="s3.internal-ceiling-height">
```

That id is what answers are stored against. Set once, when the field is created, and never changed.

- **Renaming a label is safe.** The id does not change.
- **Moving a field is safe.** Same reason.
- **Adding fields, options or sections is safe.**
- **Deleting a field still loses its data** — expected. If it is being *replaced*,

add a migration rule instead.

A control without a `data-fid` is silently excluded from saved data. No error, no warning — the answer simply never saves. `tests.html` fails if any control is missing one.

Every file input needs a permanent `data-mfid`

```
<input type="file" data-mfid="m.s4.wall-moisture-signs" id="wall-signs-media">
```

Added in v1.3. Photos were previously keyed by the input's DOM id, so renaming an id orphaned every photo attached to it — the one place the v1.2 fix had been skipped.

Naming convention

Control type	Pattern	Example
Text, select, textarea	<code>sNN.slug-of-label</code>	<code>s3.internal-ceiling-height</code>
Checkbox	<code>sNN.slug::slug-of-option</code>	<code>s3.slab-design::raft-slab</code>
Radio group	<code>r.groupname</code>	<code>r.grade</code>
"Other" free text	<code>sNN.slug-of-label-other</code>	<code>s3.retaining-wall-other</code>
File input	<code>m.sNN.purpose</code>	<code>m.s4.wall-moisture-signs</code>

Lowercase, hyphenated, derived from the label as it reads when the field is created.

Migration rules — when a field is replaced

Four mechanisms exist. Use them rather than accepting data loss:

1. `data-legacy` — put the old key on the new control to redirect its data. Multiple rules can be given, separated by `;;`.
2. `VALUE_RULES` — declared as `data-legacy="oldKey=oldValue"`. Ticks a checkbox when an old dropdown held a particular value.
3. `MERGE_RULES` — appends old free text into another field rather than overwriting, and can tick a companion checkbox.
4. `VALUE_REMAP` — rewrites a stored answer when the *wording of an option* changes.

Anything unmatched is preserved under an `unmapped:` prefix rather than discarded.

The trap. Field ids protect against renaming a **label**. They do **not** protect against rewording an **option** in a radio or checkbox group — those are stored as their text. Reword one without a `VALUE_REMAP` and the answer is orphaned on the next autosave.

The record schema number tracks this: 1 is label-keyed, 2 introduced field ids, 3 applied the BS8102:2022 grade remap, 4 added the baton pointer for handover through SharePoint. Bump it whenever a migration step is added.

`ensureSchema()` **owns that number and nothing else may set it.** `saveNow()` once pinned `cur.schema` to a literal, which was harmless until the schema moved — after which every record migrated correctly on load and was written straight back at the old number, silently, on every save. Found by accident in v1.4. If you add a migration step, grep for the number you are leaving behind before you trust it.

4. Versioning — required on every change

- `index.html`: update `APP_VER`, `VER_DATE`, and `FORM_VER` if form content changed.
- `sw.js`: bump `CACHE_VERSION`. ****This is what makes installed devices pick up the update.**** `tests.html` fails if it does not contain `APP_VER`.
- `CHANGELOG.txt`: write the entry as you make the change, not afterwards.
- If field ids changed or were added, say so explicitly — Project 3.0 maps against them.
- If any radio or checkbox option was reworded, add a `VALUE_REMAP` and bump the schema.
- Commit, and tag the release (v1.3.2).

The home-screen footer and form header read their version from these constants — no separate text edit is needed.

5. Testing

Automated. Serve the folder over HTTP and open `tests.html`. 237 assertions covering field-id integrity, the full v1.1.1 migration chain, the grade remap, media identity, update detection, naming, the upload queue, the Graph transport and sign-in. **Expect zero failures before you start and zero when you finish.**

Serving matters — `file://` blocks iframe access and IndexedDB.

Still needs a person. The tests cannot tell you whether the app *feels* right:

1. Create a test inspection and enter data in every changed field.

2. Close the tab, reopen, confirm it reloads intact — including dynamic reveals, Other boxes and the measurement schedule.
3. Export a draft, re-import it, confirm the changed fields survive.
4. Open a record created on the previous version and confirm the migration message appears and old answers are present.
5. Change the stage; confirm the audit trail grows and the exported filename picks it up.
6. Attach a photo; confirm the chip turns green and the count increments.
7. Mark complete, download the report, check the changed fields and measurement totals appear.
8. Delete the test inspection.

Development caching will bite you. The service worker serves cache-first, so an edit can appear to do nothing. Load `index.html?nosw=1` for a guaranteed fresh copy. This has twice caused the test suite to grade stale code.

6. Redeploying

The repository **is** the deployment. Pushing to `main` publishes the app.

1. Confirm tests pass and version stamps agree.
2. Commit and tag.
3. Merge `release/v1.x` into `main` and push.
4. **Verify the served file actually changed** — fetch the live `index.html` and confirm `APP_VER` moved. Not that the push succeeded; that the app did.
5. From v1.3 devices announce the update themselves. Before that, staff must fully close and reopen the app.

If devices do not update, it is almost always a missed `CACHE_VERSION` bump. Check that first. The tests now catch it, which is why the check exists.

v1.2.0 and v1.2.1 were completed and never deployed, and nobody noticed for two versions because nothing checked that live matched intent. Step 4 is that check.

7. Downstream connection (Project 3.0)

Every report contains a hidden machine-readable JSON block with all answers, section labels, the measurement schedule with totals, per-section completion counts, and the audit trail.

Action required in Project 3.0. From v1.2 the `rawData` block is keyed by permanent **field id** (e.g. `s3.slab-design`) rather than label text. The Proposal Builder must map against these ids.

`docs/FIELD-APP-TEMPLATE.md` describes the whole architecture, written so a sibling app (progress inspections, completion reports, competency assessments, equipment damage) can be built from it. Roughly 80% of the code moves across unchanged — the form is the only part that genuinely differs.

8. Ideas parked for future versions

- **SharePoint List + Power Apps** — the agreed destination for the handover problem. One record per inspection, per-field versioning, no baton discipline needed. Trigger is headcount and volume, not time.
- **B2C/B2B client type indicator** — required by 4.0 for quote type selection.
- **Required-field prompts** — better as stage-transition gates than a blocking check on *Mark complete*.
- **Update the Dryspace Guide to BS8102:2022** — the form moved in v1.2.1 and the Guide now disagrees with it.
- **SOP alignment** — Section 01 corresponds to the New Enquiry SOP, Section 02 to the Discovery Call SOP.
- **Sample photos on fields** — note that hover does not exist on touch devices.
- **App icons rebuilt from the official logo mark** — the current icons are a generic droplet.
- **Document freshness check** — compare each document's modified date against `index.html` and flag any older than the code it describes. The largest remaining manual gap in the release process.
- **Google Photos secondary backup** — scheduled sync reading from SharePoint, for its map-based search. Needs a custom connector.

Done since this list was last written

- ~~Direct SharePoint upload via Microsoft Graph~~ — built in v1.3.
- ~~The record itself moving through SharePoint~~ — built in v1.4. The share sheet survives only as the offline fallback.
- ~~Automatic backup of in-progress work~~ — built in v1.4, to `wip/`.
- **Resume a broken upload from where it stopped.** The transport creates a fresh upload session on every attempt, so a failed 5 MB photo restarts from byte 0. Persisting the session URL on the record would fix it, but it changes what the record stores, so it belongs with a schema move rather than on its own.
- **Clear out old `wip/` files.** Nothing deletes them, by design — the app never deletes from SharePoint. They accumulate one per device per inspection. Harmless but untidy; a retention job or a manual sweep at close-out.